

3D Gaussian Splatting과 RL 유도 흔들림을 이용한 사족 보행 로봇 비전 자율주행의 시각-동역학 간극 완화 파이프라인

이민석¹, 한성유¹, 문희재¹, 황성수^{1*}

¹한동대학교 AI융합교육원, 전산전자공학부,
경상북도 포항시 북구 흥해읍 한동로 558, 대한민국

{glen, tjrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * 교신저자

Abstract: 사족 보행 로봇에 비전 기반 자율주행을 배포하는 것은 시각적 sim-to-real 간극과 보행으로 인한 카메라 흔들림(jitter)으로 인해 어렵다. 이러한 흔들림은 특징점 추적과 깊이 정보의 일관성을 저하시킨다. 본 논문은 시각 및 동역학적 간극을 해소하여 시뮬레이션 상에서 파라미터를 최적화하는 파이프라인을 제안한다. 시뮬레이션 단계에서는 실제 복도를 3D Gaussian Splatting(3DGS) 디지털 트윈으로 재구성하고, 강화학습(RL) 보행 정책을 실행하여 카메라 흔들림 노이즈를 재현한다. 이러한 시각 및 동역학적 교란 조건 하에서 RTAB-Map visual SLAM과 Nav2 스택의 파라미터를 최적화한다. 실제 배포 시에는 학습된 RL 정책을 우회하고, ROS 2 브리지를 통해 속도 명령을 로봇의 내장 Sport API로 직접 전송하여 연산량을 절감한다. 실물 로봇 실험을 통해 시뮬레이션에서 최적화된 파라미터가 별도의 재조정 없이 물리 플랫폼에 직접 적용되어 비전 기반 지도작성과 장애물 회피를 수행할 수 있음을 입증한다.

Keywords: 사족 보행, Visual SLAM, Sim-to-Real 파이프라인, 3D Gaussian Splatting, Ego-motion Jitter, 가상 Sandbox.

1. 서론

자율 사족 보행 로봇은 바퀴형 시스템에 비해 험지 및 비정형 지형 통과에 우수한 이동성을 가진다. 실내 autonomy 달성을 위해서는 보행 제어와 고충실도 SLAM의 통합이 필수적이다. 전통적으로 실내 주행은 LiDAR 센서에 의존했으나, LiDAR는 무게와 전력 소모가 커 사족 보행 로봇에 제한적이다. 단일 RGB-D 카메라를 사용하는 비전 주행은 매력적인 대안이다.

그러나 비전 기반 주행 배포 시 두 가지 sim-to-real 문제가 발생한다. 첫째는 시각적 sim-to-real 간극으로, 물리 시뮬레이터의 시각적 사실성 부족이 특징점 매칭 실패를 유발한다. 둘째는 보행 유도 ego-motion noise(카메라 흔들림)로, 보행 진동이 모션 블러와 phantom obstacle을 생성한다. 고가의 사족 로봇 하드웨어에서 직접 파라미터를 튜닝하는 것은 배터리 한계와 충돌 파손 위험으로 인해 크게 제약된다.

이를 해결하기 위해, 본 논문은 단일 전방 RGB-D 카메라(D435i)를 사용하는 사족 로봇 비전 주행을 위한 실용적인 이중 단계 파이프라인을 제안한다. 제안 프레임워크는 가상 검증과 실제 배포를 분리한다. DSLR 촬영으로 3DGS를 재구성하고, 가상 카메라 파라미터를 실제 D435i와 일치시켜 센서 불일치를 차단한다. 시뮬레이션 단계에서는 3DGS 디지털 트윈과 Isaac Lab에서 학습된 RL 보행 정책을 결합하여 카메라 bobbing noise를 emulating하는 sandbox를 구축하고 RTAB-Map 및 Nav2 파라미터를 최적화한다. 실제 배포 시에는 연산 절약을 위해 RL 정책을 우회하고, ROS 2 bridge를 통해 제조사 Sport API로 명령을 직송한다.

본 연구의 기여는 다음과 같다.

1. DSLR 기반 3DGS 및 *fVDB* 재구성을 시뮬레이터에 통합하여 시각적 sim-to-real 간극을 완화하고, 가상-물리 카메라 파라미터를 동기화한 시각 검증 sandbox를 구축한다.
2. RL 보행 정책을 카메라 jitter 생성기로 활용하여 동역학적 sim-to-real 간극을 줄이고, 가상 sandbox에서 보행 진동을 재현한다.
3. 시뮬레이션 sandbox에서 최적화된 주행 및 지도작성

파라미터가 실제 하드웨어 재배포 없이 Unitree Go2에 직접 전이됨을 입증한다.

2. 관련 연구

2.1 사족 보행을 위한 강화학습

강화학습(RL)은 보행 시스템의 제어 패러다임을 크게 발전시켰다 [1]. Model Predictive Control(MPC) 및 Whole-Body Control(WBC)과 같은 전통적인 모델 기반 방법은 정확한 시스템 동역학과 환경 접촉 모델에 크게 의존한다. 반면 model-free RL 알고리즘은 시행착오 상호작용을 통해 사족 보행 플랫폼이 강건한 보행 정책을 획득할 수 있게 한다. NVIDIA Isaac Sim 및 Isaac Lab과 같은 고도로 병렬화된 물리 시뮬레이터의 등장은 수천 개의 동시 환경에서 deep RL 정책을 학습하는 것을 가능하게 하였다. 이러한 패러다임은 ANYmal 및 Unitree Go1/Go2와 같은 시스템에서 민첩한 보행의 실제 배포를 가능하게 하였다. 도메인 랜덤화와 커리큘럼 학습을 통해 연구자들은 동역학 간극을 줄였고, 정책이 시뮬레이션에서 물리 플랫폼으로 직접 전이될 수 있도록 하였다. 그러나 기존 보행 연구의 대부분은 gait 안정성과 외란 거부에 주로 초점을 맞추었으며, 고수준 비전 주행 통합과 센서 노이즈 특성은 충분히 다루지 않았다.

2.2 Visual SLAM 및 자율주행 통합

이동 로봇 주행은 높은 공간 정확도와 넓은 시야각을 가진 Light Detection and Ranging(LiDAR) 센서에 전통적으로 의존해 왔다. 그러나 LiDAR는 비용이 높고 무게와 전력 소모가 커서, payload와 전력에 제약이 있는 사족 보행 로봇에는 적합성이 낮다. RGB-D 카메라를 활용한 Visual SLAM(VSLAM)은 가볍고 비용 효율적인 대안을 제공한다. 최신 visual SLAM 프레임워크 중 Real-Time Appearance-Based Mapping(RTAB-Map) [2]은 appearance 기반 loop closure detection과 pose-graph optimization을 결합하여 drift가 최소화된 trajectory 추정을 생성할 수 있어 널리 사용된다. 동시에 ROS 2 Navigation(Nav2) 프레임워크 [3]는 경로 계획과 장애물 회

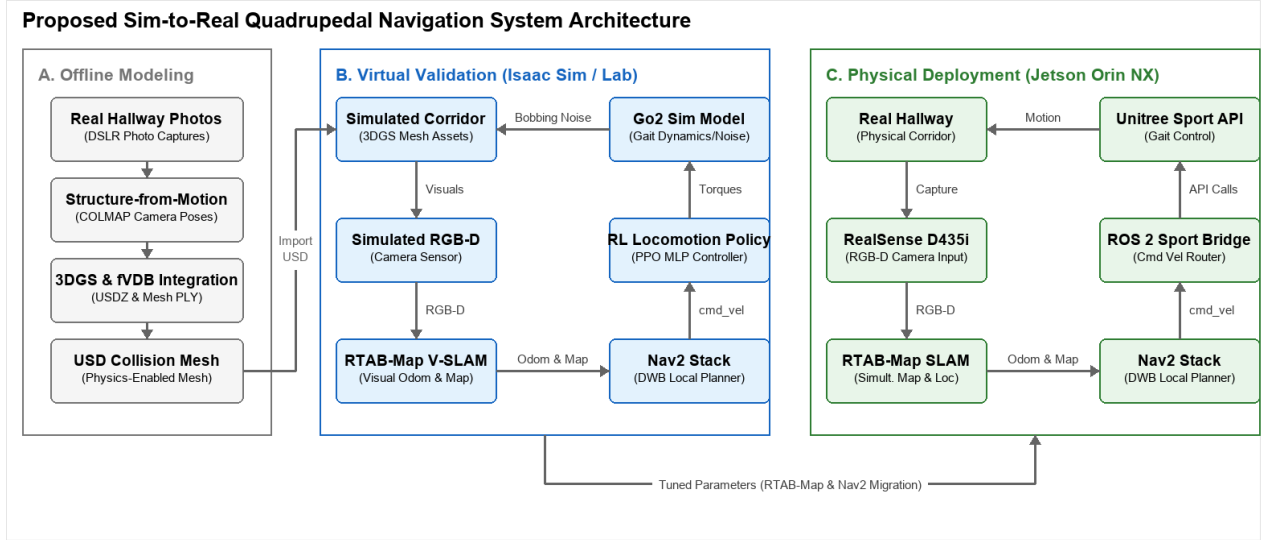


Fig. 1. 제안하는 sim-to-real 사족 보행 로봇 주행 프레임워크의 전체 구조. (a) 가상 검증 단계에서는 고충실도 3DGS 환경을 구축하고, 상위 주행 파라미터 튜닝을 위해 카메라 bobbing noise를 모사하는 RL 보행 정책을 학습한다. (b) 물리 배포 단계에서는 Jetson Orin NX의 연산 자원을 최적화하기 위해 RL 정책을 우회하고, ROS 2 bridge를 통해 Unitree Sport API로 명령을 전달한다.

피를 위한 산업 표준으로 자리 잡았다.

최근 연구들은 VR-Robo [5]와 같이 보행 및 주행을 위한 사실적인 디지털 트윈을 구축하기 위해 3DGS [4]를 활용했지만, 종종 이중 센서에 의존한다. 예를 들어 VR-Robo는 오프라인 3DGS 모델 구축을 위해 고급 소비자용 모바일 기기(iPad 또는 iPhone 등)로 환경 장면을 촬영하지만, 학습된 정책 배포에는 물리 사족 보행 로봇에 장착된 액션 카메라(Insta360 Ace 등)를 사용한다. 서로 다른 하드웨어 구성 간에 디지털 트윈을 배포하는 것은 렌즈 왜곡 모델의 불일치, 색상 프로파일 차이, 서로 다른 field-of-view(FOV) 경계와 같은 device-level 시각 센서 간극을 유발한다. 반면 제안 파이프라인은 오프라인 3DGS 재구성에는 고해상도 DSLR 카메라를 사용하고, Isaac Sim 내부의 가상 카메라를 물리 D435i RGB-D 카메라의 FOV 및 해상도와 일치하도록 설정한다. 이러한 파라미터 동기화는 가상 시각 피드백과 실제 환경 배포 사이의 불일치를 최소화한다.

3. 시스템 및 보행 제어

3.1 시스템 구조

본 프레임워크는 RTAB-Map, Nav2, 이중 제어 아키텍처를 통합한다. 시뮬레이션 환경에서는 Isaac Sim/Lab에서 PPO로 학습된 DRL 보행 정책을 통해 카메라 흔들림(bobbing noise)을 생성하고, Nav2의 속도 명령(v_{cmd})을 정책 입력으로 직접 연결한다. 실제 하드웨어 배포 시에는 연산 자원을 절약하기 위해 DRL 정책을 우회하여 속도 명령을 ROS 2 브리지를 통해 Unitree Sport API로 직접 전송한다. 이를 통해 Jetson Orin NX의 연산 부하를 저감한다. 전체 시스템 구조는 Fig. 1에 나타나 있다.

3.2 Isaac Sim에서의 보행 학습

시뮬레이션에서 카메라 흔들림을 재현하기 위해 Isaac Sim/Lab 환경에서 RL 보행 정책을 학습시킨다. 이 정책은 보행 중 발생하는 주기적 진동을 모사한다. Procedural Terrain Generator를 사용하여 비평탄 지면, 램프, 계단형 장애물을 생성하였고, 커리큘럼 학습으로 지형 난이도를 점진적으로 증가시켰다. 도메인 간극 극복을 위해 아래 Domain Randomization을 적용하였다.

- 몸통 질량 랜덤화: $\Delta m \in [-1.0, +3.0]$ kg.
- 지면 마찰 계수: $\mu \in [0.6, 0.8]$.
- 외란: base CoM에 가해지는 impulse force perturbations.

물리 주기는 200 Hz, 제어 주기는 50 Hz이다. 학습은 *skrl* 라이브러리의 PPO 알고리즘을 사용한다. Actor/Critic은 [512, 256, 128] 은닉층의 MLP 구조를 가지며, 초기 학습률 1.0×10^{-3} , 감쇄율 $\gamma = 0.99$, GAE $\lambda = 0.95$ 를 사용한다.

보합 함수는 속도 추종과 Posture 안정을 유도한다. 전체 보상 R 은 다음과 같다.

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

- $R_{lin} = \exp(-\|\mathbf{v}_{xy}^* - \mathbf{v}_{xy}\|_2^2 / \sigma_{lin}^2)$ 는 선속도 추종 보상이다 ($\mathbf{v}_{xy}^*$, $\mathbf{v}_{xy}$ 는 목표 및 실제 선속도, $w_{lin} = 1.5$, $\sigma_{lin} = 0.25$).
- $R_{ang} = \exp(-(\omega_z^* - \omega_z)^2 / \sigma_{ang}^2)$ 는 각속도 추종 보상이다 (ω_z^* , ω_z 는 목표 및 실제 yaw rate, $w_{ang} = 0.75$, $\sigma_{ang} = 0.25$).
- R_{air} 는 foot clearance air-time 보상이다 ($w_{air} = 0.25$).
- P_i 는 수직 속도, roll/pitch 진동, joint limit 편차, 과적 토크 및 급격한 joint acceleration 등을 방지하기 위한 페널티다.

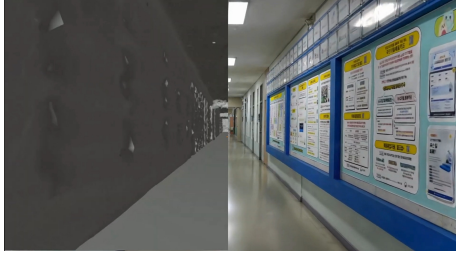


Fig. 2. 캠퍼스 복도에 대한 디지털 트윈 표현. 왼쪽: fVDB를 사용해 생성한 sparse volumetric collision mesh. 오른쪽: 고충실도 사실적 3DGS render.

3.3 3DGS 환경 구축 및 에이전트 배포

모바일 기기의 압축 왜곡을 방지하기 위해 DSLR 카메라로 촬영한 복도 이미지를 사용하였고, COLMAP의 SfM을 활용해 카메라 포즈를 추출하였다. 이를 기반으로 3DGS 모델을 최적화하여 visual USDZ 및 geometric PLY 메쉬를 생성하였으며, 물리 충돌 연산을 위해 NVIDIA의 sparse volumetric 프레임워크인 fVDB와 통합하였다 (Fig. 2).

생성된 자산을 Isaac Sim에 로드하고, fVDB representation으로 충돌 경계를 설정한다. ROS 2 브리지를 통해 가상 RGB-D 스트림을 송출하고 velocity 명령을 수신한다. 시뮬레이션 카메라 FOV와 해상도를 실제 Intel RealSense D435i와 동기화하여 시각적 차이를 줄였다. 사전 학습된 locomotion 정책은 시뮬레이션의 Go2 모델에 적용되어 장애물 회피와 복도 보행을 수행한다.

4. 자율주행

4.1 주행 시스템 구성

주행 시스템은 Intel RealSense D435i RGB-D 센서, RTAB-Map, ROS 2 Nav2 스택으로 구성된다. 시뮬레이션 환경에서도 실물 센서와 동일한 사양의 가상 카메라를 동기화하여 시각적 불일치를 축소한다. active depth 센서를 사용하여 단안 비전 기반 프레임워크(예: VR-Robo [5])에서 발생하는 깊이 추정 오류를 방지하고 일관된 로컬 costmap을 생성한다.

RTAB-Map은 RGB 스트림에서 keypoint(GFTT/ORB 등)를 추출해 오도메트리 및 루프 클로저를 연산하며, depth 데이터는 로컬 3D octomap과 2D occupancy grid를 구축한다. 장애물 감지를 위해 raw depth 프레임은 *depthimage_to_laserscan* 패키지를 통해 pseudo-2D laser scan (*/scan* 토픽)으로 변환된다. Nav2 global planner가 occupancy grid에서 글로벌 경로를 설정하면, DWB 로컬 플래너가 로컬 costmap을 참조해 속도 명령 $\mathbf{v}_{cmd}$ 를 생성한다.

시뮬레이션 검증 시 핵심은 TF 트리 불일치 해소다. Nav2는 *map* → *odom* → *base*를 요구하지만, Isaac Sim은 로봇을 *world* 좌표계에 귀속시킨다. 좌표계 충돌 방지를 위해 TF 트리를 *map* → *odom* → *world* → *base* 구조로 구성하고, RTAB-Map이 *map* → *odom*을, Isaac Sim이 *odom* → *world*를, robot state publisher가 최종 *base* 프레임을 이어붙인다. 이로써 표준 Nav2가 아무 변경 없이 시뮬레이션에서 가동된다.

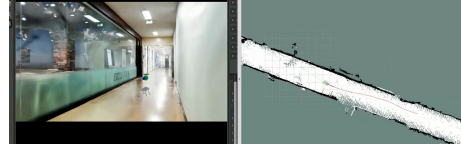


Fig. 3. 재구성된 가상 환경 내부에서의 자율주행 실행 장면. 왼쪽은 Isaac Sim 안의 시뮬레이션 사족 보행 로봇을, 오른쪽은 RViz에서의 경로 계획 및 costmap 동기화를 보여준다.

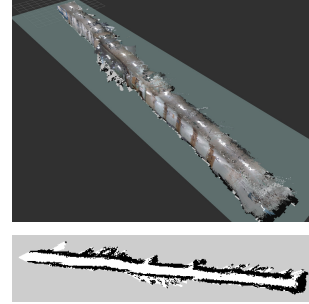


Fig. 4. 가상 sandbox에서의 visual mapping 결과. 위: RTAB-Map으로 재구성한 3D point cloud map. 아래: 투영된 2D occupancy grid map.

4.2 3DGS 환경에서의 검증

실제 배포 전, 3DGS 기반 복도(길이 80 m, 폭 2.5 m)에서 자율주행 성능을 사전 검증하였다.

시뮬레이션 환경에서 보행 정책은 고유수용성 감각(joint state, base angular velocity)과 3DGS 충돌 메쉬 조화를 통한 외수용성 고도(height-scan) 정보를 입력받아 보행 안정을 유지하며, Nav2 DWB 플래너로부터 목표 속도를 수급한다 (Fig. 3).

장애물 회피 성능 평가를 위해 가상 장애물을 배치하였다. Local planner는 장애물 주변으로 우회 경로를 계산하였고, 보행 정책은 보행 패턴을 조정하며 충돌 없이 통과하였다. 최종 3D 포인트 클라우드 및 2D occupancy grid 맵은 Fig. 4과 같다.

4.3 물리 로봇 배포 및 결과

시뮬레이션 검증 이후, 제안하는 파이프라인을 실물 Unitree Go2 사족 보행 로봇에 배포하였다 (Fig. 5). 온보드 하드웨어인 NVIDIA Jetson Orin NX (20W 모드, 16GB RAM)는 Ubuntu 20.04 및 ROS 2 Foxy를 실행한다. 연산 자원 제약으로 인해 실물 배포 시에는 RL 보행 정책을 우회하고, 속도 명령을 ROS 2 브리지를 통해 로봇 내장 Sport API로 전송하였다. 이를 통해 Jetson Orin NX의 연산 부담을 해소하여 RTAB-Map visual SLAM 및 Nav2 경로 계획 처리에 집중시켰다. RealSense D435i 센서는 USB 3.0을 통해 온보드 컴퓨터에 연결되었다.

시뮬레이션(Ubuntu 24.04, ROS 2 Jazzy)과 물리 로봇(Ubuntu 20.04, ROS 2 Foxy) 간의 OS 및 ROS 2 버전 불일치 문제를 해결하기 위해, visual 주행 스택을 ROS 2 Foxy로 마이그레이션하여 Jetson Orin NX에서 구동시켰다. 버전 차이에도 불구하고 시스템 아키텍처가 동일하게 유지되어 시뮬레이션 상에서 카메라 흔들림 조건하에 최적화된 파라미터(costmap inflation radius, voxel grid resolution, visual loop-closure matching threshold)를 물리 로봇에 직접 적용할 수 있었으며, 현장 재조정



Fig. 5. 실제 환경 실험 구성. Unitree Go2 로봇에는 Intel RealSense D435i와 Jetson Orin NX가 장착되어 있으며, 전체 navigation stack은 온보드로 실행된다.

없는 sim-to-real 전이를 달성하였다.

RTAB-Map은 물리 환경에서 고주파 카메라 진동을 보상하며 루프 클로저 감지를 성공적으로 수행하였다. Nav2는 안정적인 글로벌 경로를 유지하였으며, 로봇은 목표 속도 0.4 m/s로 주행을 지속했다. 이 결과는 시뮬레이션 흔들림 조건 하에 도출된 visual SLAM 및 costmap 파라미터가 물리 하드웨어에 유효하게 전이됨을 실증한다.

5. 결론

본 논문은 사족 보행 로봇을 위한 이중 단계 비전 주행 프레임워크를 제안하였다. 3D Gaussian Splatting 디지털 트윈 복도와 RL 보행 정책을 이용해 카메라 흔들림을 시뮬레이션함으로써 visual SLAM 및 경로 계획 검증을 위한 가상 환경을 구축하였다. 물리 배포 시에는 RL 정책을 우회하고 ROS 2 브리지를 통해 로봇 내장 Sport API로 직접 주행 명령을 라우팅하여 온보드 컴퓨터의 연산 효율을 높였다. 실물 Unitree Go2 로봇을 이용한 실험을 통해, 시뮬레이션 환경에서 흔들림 노이즈를 반영하여 도출된 파라미터가 실물 하드웨어에 재보정 없이 유효하게 이식됨을 검증하였다.

향후 연구는 동적 장애물 회피 전략을 통합하고, 엣지 컴퓨터에서 더 높은 해상도의 시각 피드백을 지원하도록 처리 파이프라인을 최적화하는 데 초점을 둘 것이다.

감사의 글

이 연구는 과학기술정보통신부의 재원으로 한국연구재단(NRF)의 지원을 받아 수행되었다(No. RS-2025-24683458).

REFERENCES

- [1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.
- [2] M. Labbé and F. Pomerleau, “Online global loop closure detection for large-scale multi-session graph-based SLAM,” *Autonomous Robots*, vol. 34, no. 3, pp. 183–200, 2013.
- [3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette,

and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

- [4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Dretakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
- [5] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, “VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion,” *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 7875–7882, August 2025.